

4. Phase 2 ADS PSVセマンティック結合：環境定義、変換操作及び関数単位の処理

この文書は、**src/semantic_binding.py** を対象とし、Phase 1構造化CSVからADS PSVを生成する処理を説明します。同じ変換エンジンを使用するISO 21378 ADCのCSV出力も、この文書の付録として扱います。

ADS XBRL GLはXML構文を生成する別の構文結合処理であるため、[05_PHASE2_ADS_XBRL_GL_SYNTAX_BINDING.md](#)へ分離しました。

処理の流れ

```
Structured CSV + semantic binding CSV
→ C行から繰返しクラスを登録
→ A行のsemantic_pathから元列を導出
→ [n]及び行スコープを解決
→ インボイス単位又は繰返しクラス単位で値を統合
→ 結合表順のPSV / CSVを出力
```

ADS PSV及びフラットファイル変換仕様

1. 目的

この文書は、UADC概念実証（PoC）で使用するセマンティック結合変換プログラムを規定します。

このプログラムは、セマンティック結合CSVを適用し、Phase 1で生成したUADC構造化CSVを、Phase 2の対象フラットファイルへ変換します。対象フラットファイルには、パイプ区切りのPSV又はカンマ区切りのCSVを使用できます。ADS PSVファイルとISO 21378 ADCインボイスCSVファイルには、同じ処理モデルを適用します。

実装レベルの処理の詳細は、この文書の第7章、第8章及び第15章に記載しています。

この文書に示すパスは、すべて **UADC_PoC** 作業ディレクトリからの相対パスです。

2. メインプログラム

プログラム：

```
src/semantic_binding.py
```

この変換プログラムは、XML構文に依存しません。構造化CSVの行を読み込み、**semantic_path** に従って値を対象列へ対応付けます。

3. 入力ファイル

3.1 構造化CSV入力

入力には、**src/syntax_binding.py** が生成したPhase 1構造化CSVファイルを使用します。

例：

```
out/phase1/openpeppol_ubl_invoice_minimal.csv
```

複数の構造化CSVファイルを含むディレクトリを入力として指定することもできます。その場合は、ディレクトリ内の各 **.csv** ファイルを個別に変換します。

3.2 セマンティック結合CSV

セマンティック結合CSVは、次のディレクトリにあります。

```
specs/bindings/semantic/
```

現在のADS用ファイルには、次のものがあります。

```
ADS_Invoices_Received_PSV_Binding.csv
ADS_Invoices_Generated_PSV_Binding.csv
ADS_Invoices_Received_Lines_PSV_Binding.csv
ADS_Invoices_Generated_Lines_PSV_Binding.csv
ADS_Supplier_Listing_PSV_Binding.csv
ADS_Customer_Master_PSV_Binding.csv
```

現在のISO 21378:2019 ADCインボイス用ファイルには、次のものがあります。

```
ISO21378_SAL_Invoice_Generated_CSV_Binding.csv
ISO21378_SAL_Invoice_Generated_Details_CSV_Binding.csv
ISO21378_PUR_Invoice_Received_CSV_Binding.csv
ISO21378_PUR_Invoice_Received_Details_CSV_Binding.csv
```

これらのファイルは、表38、表39、表53及び表54のフラットなインボイスビューを実装します。繰り返されるTAXグループは、**Tax1** から **Tax4** までの列へ展開します。繰り返されるBUSINESS SEGMENTの値は、**Business_Segment_01** から **Business_Segment_05** までの列で表します。CSVのヘッダーに同じフィールド名を繰り返して記載することはできないため、このように物理的に展開する必要があります。

4. 結合表の仕様

セマンティック結合表は、対象定義表を基礎として、UADCの対応付けに必要な列を追加します。

```
field_no,field_name,level,flat_file_data_type,length,description,source_document,semantic_path,type,multiplicity,mapping_status,mapping_note
```

4.1 A行

type=A の行は、対象ファイルの列を出力します。

主要なフィールドは、次のとおりです。

- **field_no** : 出力列の順序を制御します。
- **field_name** : 出力する対象列名です。
- **semantic_path** : UADC構造化CSV上の値を識別します。
- **description** : 対象定義文書から取得した説明です。
- **mapping_status** : **direct**、**approximate**、**requires_transformation** 又は **not_available** のいずれかです。
- **mapping_note** : 意味上の近似又はデータ不足の内容を説明します。

変換プログラムは、**semantic_path** の最後のセグメントから、構造化CSVの元列を導出します。

例 :

```
$.invoice.invoiceNumber -> InvoiceNumber  
$.invoice.seller.sellerIdentifier -> SellerIdentifier
```

4.2 C行

type=C の行は、行スコープの解決に使用するUADCセマンティッククラスを定義します。

主要なフィールドは、次のとおりです。

- **field_name** : セマンティッククラス名です。
- **description** : 構造化CSVモデルの定義から複写した説明です。
- **semantic_path** : セマンティッククラスを識別します。
- **multiplicity** : そのクラスを繰り返すことができるかどうかを示します。

多重度の上限が *****、**n**、**unbounded** 又は2以上であるクラスを、繰返しクラスとして扱います。

変換プログラムは、クラスのセマンティックパスから、構造化CSVのディメンション列を導出します。

例 :

```
$.invoice.invoiceLine -> dInvoiceLine
```

5. 繰返し行スコープ

変換プログラムは、インデックスが指定されていない **A** 行が使用する繰返し **C** 祖先のうち、最も深いクラスを対象行スコープとして選択します。

インボイスレベルの対象には、繰返し行スコープがありません。変換プログラムは、疎な構造化 CSV 行を統合し、インボイスごとに1件の対象行を生成します。

明細レベルの対象では、**\$.invoice.invoiceLine** を行スコープとして使用します。変換プログラムは、**dInvoiceLine** の各発生に対して1件の対象行を出力し、それぞれの明細行に親インボイスの値を統合します。

VAT内訳又は値引きなど、その他の繰返しクラスについても、セマンティック結合表がそれらをインデックスなしの繰返し祖先として使用する場合は、同じ規則を適用します。

6. インデックス付き繰返し値

繰返しグループを対象列へ横方向に展開する場合は、0を起点とするインデックス付きセマンティックパスを使用します。

例：

```
$.invoice.vatBreakdown[0].vatCategoryCode  
$.invoice.vatBreakdown[1].vatCategoryCode  
$.invoice.vatBreakdown[2].vatCategoryCode
```

インデックス付きパスは、出力行を追加しません。現在のインボイス又は現在の繰返し行コンテキストの中から、指定した発生を選択します。

ISO 21378のヘッダー結合では、**Tax1** から **Tax4** までを出力するために、インデックス付きのVAT内訳パスを使用します。ADCのヘッダーフィールドは単一値であるため、インボイス注記及び支払指示にも発生番号0を使用します。明細結合では、繰返し **InvoiceLine** クラスを出力行スコープとして使用します。EN 16931では、インボイス明細ごとに一つのVAT分類を定義するため、明細行では **Tax1_Type_Code** だけを直接対応付けることができます。

7. 内部データ構造

read_csv_rows(path) は、次の値を返します。

```
list[dict[str, str]]
```

各辞書は、ヘッダー名をキーとするCSVの1行を表します。

SemanticClass は、次の情報を保持します。

- **semantic_path**
- **multiplicity**

繰返し **SemanticClass** 行は、次の形式で収集します。

```
dict[str, SemanticClass]
```

SemanticBinding は、対象列一つ分の結合情報として、次の項目を保持します。

- **sequence**
- **target_column**
- **semantic_path**
- **normalized_semantic_path**
- **source_column**
- **repeat_group_path**
- **repeat_group_column**
- **repeat_index**
- **default_value**
- **required**

load_bindings(binding_csv) は、次の値を返します。

```
list[SemanticBinding]
```

このリストは、**field_no** 又は **sequence**、続いて対象列名の順に並べ替えられます。

8. 処理関数

load_bindings(binding_csv) は、結合表を読み込み、繰返し **C** クラスを記録し、**A** 行から **SemanticBinding** オブジェクトを作成します。また、セマンティックパスから元列を導出し、結合表に基づいて繰返し行スコープを解決します。

row_scope_group(bindings) は、対象ビューが使用する最も深い繰返しグループを選択します。これにより、出力がインボイスレベルか、繰返し行レベルかが決まります。

transform_rows(rows, bindings) は、構造化CSV行に結合表を適用します。繰返し行スコープが存在しない場合は、疎な行を統合してインボイスレベルの対象行を作成します。

transform_repeated_group_rows(rows, bindings, scope_path, scope_column) は、**dInvoiceLine** ごとなど、繰返しグループの各発生に対して1件の対象行を出力します。

row_repeat_indices(source_row, bindings, repeat_counts) は、インデックス付き繰返しパスに0を起点とする発生番号を割り当てます。

merge_values(target_row, source_row, bindings, repeat_indices) は、構造化CSVの空でない値を対象列へ複写します。対象列ごとに、最初に見つかった空でない値を使用します。

output_path(input_csv, output_dir, extension, output_filename, binding_csv) は、単一入力ファイルに対して **--output-filename** が指定されていない限り、次のパスに出力します。

```
out/phase2/<target-family>/<structured-csv-stem>/<target-view>.<extension>
```

9. 出力仕様

出力ヘッダーは、セマンティック結合CSVにおける **A** 行の順序に従います。

既定の出力形式は、次のとおりです。

形式	区切り文字	拡張子
psv		.psv
csv	,	.csv

対象ビューのファイル名部分は、結合ファイル名から導出します。

```
ADS_Invoices_Received_PSV_Binding.csv -> Invoices_Received.psv
ADS_Customer_Master_PSV_Binding.csv -> Customer_Master.psv
ISO21378_PUR_Invoice_Received_CSV_Binding.csv -> PUR_Invoice_Received.csv
```

10. 対象外の機能

- XMLを解析しません。
- XPathを使用しません。
- 対象列名だけから対応関係を推定しません。
- 実行時に外部のLHM CSVを読み込みません。
- xBRL-CSVメタデータJSONを生成しません。
- 会計カレンダーがなければ、会計年度又は会計期間を導出しません。
- 明細総額又は明細税額を計算しません。
- EN 16931に存在しないERP監査証跡、転記勘定、状態又は追加の事業セグメント値を作り出しません。

11. 運用手順

Phase 2では、Phase 1構造化CSVと対象のセマンティック結合表を読み込みます。必要な対象ビューに対応する結合表を選択して変換プログラムを実行し、**out/phase2/** 以下に生成されたファイルを確認します。すべての対応関係は、**field_name**、**semantic_path**、クラスの **multiplicity** 及び結合表の順序に基づいて解決します。

結合表の **type=C** 行がクラススコープを確立し、**type=A** 行が対象フィールドを定義します。変換プログラムは、セマンティックパスの最後のセグメントから構造化CSVの元列を導出し、各フィールドを最も深い繰返しクラスの祖先へ割り当てます。

12. ADS PSV

ADS PSVは、**src/semantic_binding.py** に **psv** プリセットを指定して生成します。6種類のADS結合ビューは、PoCで必要とするインボイス及びマスタデータの観点を対象とします。各出力ヘッダーは結合表の順序に従い、繰返される対象レコードは、結合表で選択した繰返しセマンティッククラスに従います。

ERPのユーザー操作履歴、転記状態、仕訳勘定又は期間締め処理に依存するフィールドは、EN 16931に元の値がない場合、空欄のままとします。これは明示された元データの境界であり、ゼロ又は既定値を推定したものではありません。

14. ISO 21378 ADC

ISO 21378 ADCの実装では、4種類のインボイス用セマンティック結合表と、**src/semantic_binding.py** のCSVプリセットを使用します。これにより、同じ構造化CSVから、独立して設計された会計データインターフェースを生成できることを示します。

対応範囲は、インボイスから取得できるフィールドに限られます。取引の承認、ユーザー操作履歴、転記明細、締め処理情報及び報告書作成の状態には、ERPの元データが必要です。したがって、生成するADCファイルには、既知のインボイス値と明示的な不足項目を保持し、存在しない会計記録を作り出しません。

15. 関数単位の処理仕様

15.1 ADS及びADCの区切り文字形式への変換

- **load_bindings** : クラス行及び属性行を順番に読み込み、必須対象フィールドを検証します。
- **indexed_semantic_path** : 明示的にインデックスが指定された繰返し値を認識します。
- **source_column_for_path** : セマンティックパスの最後のセグメントを構造化CSV列へ対応付けます。
- **dimension_column_for_path** : クラスパスに対応するディメンションを特定します。
- **deepest_repeated_class_ancestor** 及び **resolve_bindings_for_classes** : 各対象フィールドを、その実効的な繰返し行スコープへ割り当てます。
- **transform_rows** : 現在の祖先値を保持し、対象行を作成します。
- **transform_repeated_group_rows** : 個別にインデックスを付けた繰返しレコードを出力します。
- **merge_values** : 現在の行が所有し、対象結合で必要とされる値だけを複写します。
- **write_target_file** : 所定の順序でPSV又はCSVのヘッダー及びレコードを書き出します。

単一の子クラスは、現在の繰返し祖先へ統合します。繰返し子クラスは、最初の発生から独立した対象レコードを作成します。この動作は、**02_STRUCTURED_CSV_LHM_BINDINGS.md** に規定する構造化CSVの仕様と一致します。

16. 検証及びトラブルシューティング

代表的なインボイス一つだけでなく、Phase 1のすべての例について対象ファイルを検証してください。ADS XBRL GLの検証は **05_PHASE2_ADS_XBRL_GL_SYNTAX_BINDING.md** に分離しています。ADS PSV及びADCについては、ヘッダー順、レコード件数、区切り文字及び代表的な繰返し値を確認します。

フィールドが出力されない場合は、次の順序で確認してください。

1. Phase 1構造化CSVに値が存在すること。
2. セマンティックパスの最後のセグメントが、想定する元列名であること。
3. 必要なすべての **type=C** 祖先及び多重度が結合表に存在すること。
4. 結合表の統合処理でセクタコンテキストが保持されていること。
5. 対象パス及び対象フィールド名が設定されていること。
6. その行が、対象ビューで想定する繰返しスコープに属していること。

17. セマンティック結合変換の概要

セマンティック結合変換文書

この文書では、構造化CSVを対象フラットファイルへ変換するプログラムを説明します。

セマンティック結合変換プログラムは、Phase 1 UADC構造化CSVとセマンティック結合CSVを読み込み、ADS PSV又はISO 21378 ADC CSVなどのPhase 2対象フラットファイルを書き出します。結合表は、UADCのどの **semantic_path** の値をどの対象列へ出力するか、また、どのセマンティッククラスが繰返し対象行を制御するかを定義します。

ISO 21378:2019 ADCインボイス結合は、現在、表38、表39、表53及び表54を対象としています。出力するCSVのヘッダーをすべて一意にするため、対象TAXグループは4組の番号付き列へ、BUSINESS SEGMENTは5組の番号付き列へ展開します。各対象行には、**mapping_status** 及び **mapping_note** も記録します。これらの列により、直接対応、意味上の近似、変換が必要な項目及びEN 16931インボイスから取得できない元データを区別します。

これら4種類のビューにより、計画したPhase 2 PoC基準範囲のISO 21378部分が完成します。ここでいう完成とは、対象定義、結合定義、CSV生成、回帰テスト及び明示的な不足分類が揃っていることを意味します。EN 16931の元インボイスだけから、ISO 21378のすべてのデータを完全に取得できることを意味するものではありません。

内部の **dict/list/dataclass** オブジェクト及び関数単位のデータフローなど、より広い処理モデルと共通する実装レベルの詳細については、**02_STRUCTURED_CSV_LHM_BINDINGS.md** 第15章を参照してください。

ファイル

- **program_specification.md** : 変換プログラムの入力、出力、セマンティック結合行、繰返し行スコープ、インデックス付き繰返し値、出力ファイル名及び対象外の機能を定義します。
- **user_guide.md** : ADS PSV及びCSVの生成、ディレクトリ入力並びにトラブルシューティングのコマンド例を示します。

- **iso21378_adc_invoice_coverage.md** : ISO 21378 ADCインボイスの対応範囲、データ不足及びPhase 1回帰テストの結果を記録します。

関連ディレクトリ

- **../src/** : 変換プログラムの実装。特に **semantic_binding.py**。
- **../specs/bindings/semantic/** : 現在使用しているセマンティック結合CSV。
- **../specs/bindings/syntax/** : セマンティックな元データとなるPhase 1構造化CSVモデルを定義する構文結合CSV。
- **../out/phase1/** : Phase 1構造化CSV入力。
- **../out/phase2/** : 生成されたPhase 2対象出力。

Phase 2のセマンティック結合は、共通の構造化CSVを起点とします。XMLを直接解析せず、元XMLのXPath式にも依存しません。

18. セマンティック結合ユーザーガイド

ユーザーガイド : セマンティック結合による構造化CSVからフラットファイルへの変換

1. 作業ディレクトリ

コマンドは、**UADC_PoC** ディレクトリで実行します。

```
cd UADC_PoC
```

Windows PowerShellでは、Pythonコマンドを次のように設定します。

```
$python = 'python'
```

macOS又はLinuxのシェルでは、次のように設定します。

```
PYTHON=python3
```

2. メインスクリプト

次のスクリプトを使用します。

```
src/semantic_binding.py
```

このスクリプトは、セマンティック結合CSVを使用して、Phase 1構造化CSVをPhase 2対象PSV又はCSVファイルへ変換します。

3. ADS Invoices Received PSVの生成

次のコマンドを実行します。

```
& $python .\src\semantic_binding.py `
.\out\phase1\openpeppol_ubl_invoice_minimal.csv `
-b .\specs\bindings\semantic\ADS_Invoices_Received_PSV_Binding.csv `
-o .\out\phase2\ADS_PSV `
--format psv
```

出力：

```
out/phase2/ADS_PSV/openpeppol_ubl_invoice_minimal/Invoices_Received.psv
```

4. ADS Invoices Received Lines PSVの生成

次のコマンドを実行します。

```
& $python .\src\semantic_binding.py `
.\out\phase1\openpeppol_ubl_invoice_minimal.csv `
-b .\specs\bindings\semantic\ADS_Invoices_Received_Lines_PSV_Binding.csv `
-o .\out\phase2\ADS_PSV `
--format psv
```

出力：

```
out/phase2/ADS_PSV/openpeppol_ubl_invoice_minimal/Invoices_Received_Lines.
psv
```

この対象には、繰返し行スコープがあります。結合表には、繰返し多重度を持つ **\$.invoice.invoiceLine** の **type=C** 行が含まれるため、変換プログラムは **dInvoiceLine** の各発生に対して1行を出力します。

5. Phase 1構造化CSVの一括変換

次のコマンドを実行します。

```
& $python .\src\semantic_binding.py `
.\out\phase1 `
-b .\specs\bindings\semantic\ADS_Invoices_Received_PSV_Binding.csv `
-o .\out\phase2\ADS_PSV `
--format psv
```

入力する構造化CSVのファイル名部分を使用して、各入力CSVに一つの出力ディレクトリを作成します。

例：

```
out/phase2/ADS_PSV/Allowance-example/Invoices_Received.psv
```

6. PSVではなくCSVを生成する

--format csv を指定します。

```
& $python .\src\semantic_binding.py `
.\out\phase1\openpeppol_ubl_invoice_minimal.csv `
-b .\specs\bindings\semantic\ADS_Invoices_Received_PSV_Binding.csv `
-o .\out\phase2\ADS_CSV `
--format csv
```

出力：

```
out/phase2/ADS_CSV/openpeppol_ubl_invoice_minimal/Invoices_Received.csv
```

7. ISO 21378 ADCインボイスCSVの生成

ISO 21378:2019表53の仕入インボイス・ヘッダー結合を使用します。

```
& $python .\src\semantic_binding.py `
.\out\phase1\bis-billing3-examples\Allowance-example.csv `
-b
.\specs\bindings\semantic\ISO21378_PUR_Invoice_Received_CSV_Binding.csv `
-o .\out\phase2\ISO21378_ADC `
--format csv
```

出力：

```
out/phase2/ISO21378_ADC/Allowance-example/PUR_Invoice_Received.csv
```

表54には **ISO21378_PUR_Invoice_Received_Details_CSV_Binding.csv** を使用します。表38及び表39には、対応する **SAL** ファイルを使用します。

出力を完全なADC提出データとして扱う前に、各結合表の **mapping_status** を確認してください。

- **direct** : UADCの値をそのまま複写できます。
- **approximate** : 最も近いEN 16931の値を複写しますが、意味は完全には一致しません。
- **requires_transformation** : 計算、解析規則、コード変換又は会計カレンダーが必要です。
- **not_available** : 必要なERP又は監査データがEN 16931インボイスに含まれていません。

8. コマンドオプション

基本形式 :

```
& $python .\src\semantic_binding.py INPUT `
-b BINDING_CSV `
-o OUTPUT_DIR `
--format psv
```

オプション :

- **INPUT** : 一つの構造化CSVファイル又は構造化CSVファイルを含むディレクトリ。
- **-b、--binding-csv** : セマンティック結合CSV。
- **-o、--output-dir** : 出力ディレクトリ。
- **--format** : 出力プリセット。 **psv** 又は **csv**。
- **--delimiter** : 区切り文字を明示的に指定します。 **--format** の指定より優先されます。
- **--extension** : 出力拡張子を明示的に指定します。 **--format** の指定より優先されます。
- **--output-filename** : 単一ファイル入力に対する出力ファイル名を明示的に指定します。

9. 結合表の確認事項

対象フィールドについて :

- **type=A** を使用する。
- 出力する対象列名を **field_name** に設定する。
- UADCの元データを **semantic_path** に設定する。
- **field_no** を対象ファイルの出力順に設定する。

行スコープのクラスについて :

- **type=C** を使用する。
- セマンティッククラスのパスを **semantic_path** に設定する。
- 構造化CSVモデルの説明を **description** に設定する。

- クラスの多重度を **multiplicity** に設定する。

横方向に展開する繰返し値について：

- **\$.invoice.vatBreakdown[0].vatCategoryCode** のように、0を起点とするインデックス付きパスを使用する。
- 同じ結合表に、繰返しクラスを **type=C** 行として含める。

10. トラブルシューティング

出力列が空欄の場合は、**semantic_path** の最後のセグメントをUpperCamelCaseへ変換した名前が、構造化CSVの元列名と一致することを確認してください。

明細レベルのファイルが1行しかない場合は、結合表に **\$.invoice.invoiceLine** の繰返し **type=C** 行があり、多重度が **1..*** 又は **0..*** になっていることを確認してください。

対象ファイル名が想定と異なる場合は、結合CSVのファイル名を確認してください。変換プログラムは、**ADS_Invoices_Received_PSV_Binding.csv** などの名前から対象ビュー名を導出します。

19. ISO 21378 ADCインボイスの対応範囲

ISO 21378 ADCインボイスのセマンティック結合対応範囲

適用範囲

Phase 2のセマンティック結合は、ISO 21378:2019 Audit Data Collectionの次のインボイスビューを対象とします。

ADC 表	対象ビュー	結合ファイル
表38	SAL Invoice Generated	ISO21378_SAL_Invoice_Generated_CSV_Binding.csv
表39	SAL Invoice Generated Details	ISO21378_SAL_Invoice_Generated_Details_CSV_Binding.csv
表53	PUR Invoice Received	ISO21378_PUR_Invoice_Received_CSV_Binding.csv
表54	PUR Invoice Received Details	ISO21378_PUR_Invoice_Received_Details_CSV_Binding.csv

結合定義は、ISO 21378のフラットなインボイスモデルを使用します。繰り返されるTAXグループは4組の番号付き列へ、BUSINESS SEGMENTは5組の番号付き列へ展開します。

この範囲により、PoCのPhase 2は完成とします。完成の判定条件は、対象定義のレビュー、実行可能な結合定義、現在のすべてのPhase 1入力に対するCSV出力及び直接複写できないフィールドの明示的な分類が揃っていることです。EN 16931の元モデルにはERPの監査証跡及び転記データが含まれないため、ISO 21378の全フィールドをすべて設定することは、完成の判定条件ではありません。

対応状況

対象ビュー	フィールド数	直接対応	近似対応	変換が必要	取得不可	結合済みパス
PUR Invoice Received	42	17	5	5	15	22
PUR Invoice Received Details	39	12	1	2	24	13
SAL Invoice Generated	41	17	4	5	15	21
SAL Invoice Generated Details	39	11	1	2	25	12

結合済みパス は、直接対応と近似対応の合計です。現在のセマンティック結合変換プログラムは値を複写するだけで、計算又は解析を行わないため、変換が必要な項目は結合済みとして数えません。

データ充足性の評価

Phase 1のEN 16931構造化CSVには、主要なインボイスデータを投影するために十分な情報があります。インボイス識別子、日付、取引当事者、通貨、合計、支払情報、VAT内訳、インボイス明細、数量、価格、品目識別子及び購買又は販売注文への参照を取得できます。

一方、完全なISO 21378 ADCデータを作成するには不十分です。主な不足項目は、次のとおりです。

- 会計カレンダーを必要とする会計年度及び会計期間
- 解析又は専用の元フィールドを必要とする、正規化された支払条件の構成要素
- 作成、承認及び最終変更に関するERP操作記録
- インボイスのライフサイクル状態
- 総勘定元帳の借方、貸方及び税額転記勘定
- 計算又は配賦を必要とする明細総額及び明細税額
- 個別の基本単位数量及び単位コード
- EN 16931の単一の会計参照を超える複数の事業セグメント値
- EN 16931では明細ごとに一つのVAT分類を定義するため、一つのインボイス明細に対する複数の税構造

したがって、現在の結合定義は、Phase 2の概念実証用データの生成及び対応不足の分析には使用できませんが、ISO 21378 ADCの完全なデータが生成できると主張するためのものではありません。

テスト結果

4種類の結合定義を、現在のPhase 1構造化CSVインボイスファイル10件すべてに適用し、40件の対象ファイルを生成しました。

対象ビュー	ファイル数	データ行合計	ファイル当たり最小行数	ファイル当たり最大行数
PUR Invoice Received	10	10	1	1
PUR Invoice Received Details	10	18	1	3
SAL Invoice Generated	10	10	1	1
SAL Invoice Generated Details	10	18	1	3

4種類の結合定義に含まれる空でないすべてのセマンティックパスについて、**EN16931_CIUS_Invoice_LHM.csv** と照合しました。結合済みのすべてのパスが、現在のUADC LHMに存在します。

semantic_binding.py の詳細処理

Phase 2セマンティック結合：構造化CSVからPSV又はCSVへ

スクリプト：

```
src/semantic_binding.py
```

主な関数：

```
load_bindings(...)
transform_rows(...)
transform_repeated_group_rows(...)
merge_values(...)
convert_file(...)
```

入力：

- Phase 1構造化CSV
- **specs/bindings/semantic/** 配下のセマンティック結合CSV

出力：

- ADS PSV又はCSV形式の出力ファイル

セマンティック結合表

セマンティック結合表は、出力先の定義列にUADCの対応付け列を追加した構成です。

```
field_no,field_name,level,flat_file_data_type,length,description,source_document,semantic_path,type,multiplicity
```

出力項目行は **type=A**、クラス行は **type=C** とします。クラス行は出力列にはなりません。どのセマンティッククラスが繰り返すかをスクリプトへ伝えます。

実行時オブジェクト

read_csv_rows(path) は、次を返します。

```
list[dict[str, str]]
```

load_bindings(binding_csv) は、次を構築します。

```
repeated_classes: dict[str, SemanticClass]
bindings: list[SemanticBinding]
```

SemanticClass は、次を保持します。

- **semantic_path**
- **multiplicity**

repeated_classes には、繰り返しクラス行だけを保持します。

SemanticBinding は、次を保持します。

- **target_column**
- **semantic_path**
- **normalized_semantic_path**
- **source_column**
- **repeat_group_path**
- **repeat_group_column**
- **repeat_index**

入力列の導出

入力構造化CSV列は、**semantic_path** の最後のセグメントから導出します。

```
$.invoice.invoiceNumber -> InvoiceNumber
$.invoice.invoiceLine.invoiceLineIdentifier -> InvoiceLineIdentifier
```

添字付きパスは、先に正規化します。

```
$.invoice.vatBreakdown[0].vatCategoryTaxAmount
-> 正規化パス: $.invoice.vatBreakdown.vatCategoryTaxAmount
-> 入力列: VatCategoryTaxAmount
-> 繰返しグループパス: $.invoice.vatBreakdown
-> 繰返しグループ列: dVatBreakdown
-> 繰返し添字: 0
```

セマンティックディメンションの所有関係

セマンティック結合でも、構文結合と同じ三つの結合属性、すなわち **type=C**、**multiplicity** 及び **semantic_path** からファクトの所有関係を解決します。

1. **load_bindings(...)** が **read_csv_rows(...)** ですべての行を読み込む。
2. 各 **type=C** 行について、**multiplicity_repeats(...)** が繰返し行スコープを作るクラスか判定する。単一クラスは構造化CSVの独立したディメンション行を作らないため、繰返しクラスだけを **repeated_classes** に格納する。
3. **indexed_semantic_path(...)** が、**[0]** などの明示的な出現指定を除去しつつ、0始まりの **repeat_index** を保持する。
4. **source_column_for_path(...)** が、最後のセマンティックパスセグメントを構造化CSVのファクト列へ変換する。
5. **deepest_repeated_class_ancestor(...)** が、各ファクトの上位にある最も近い繰返しクラスを取得する。
6. **dimension_column_for_path(...)** が、そのクラスパスを **dXxx** 列へ変換する。
7. **resolve_bindings_for_classes(...)** が、結果を **SemanticBinding.repeat_group_path** 及び **SemanticBinding.repeat_group_column** に保存する。

したがって、非繰返しの子クラス配下のファクトは、最も近い繰返し祖先へ割り当てられます。繰返し子クラス配下のファクトは、1番目からその子自身のディメンションへ割り当てられます。この所有関係は、どの疎な構造化CSV行がファクトの入力になれるかを制御しますが、親ファクトを構造化CSVの子行へ複写するものではありません。

行スコープの選択

resolve_bindings_for_classes(bindings, repeated_classes) は、各 **SemanticBinding** に行スコープ情報を設定します。添字がないパスについて、結合表にある最も深い繰返し **C** 祖先を検索します。

例：

```
semantic_path: $.invoice.invoiceLine.invoiceLineNetAmount
repeated class: $.invoice.invoiceLine
repeat_group_column: dInvoiceLine
```

row_scope_group(bindings) は、出力ファイルが繰返し出現ごとに一行を生成するかどうかを決定します。

規則：

1. 添字付きの結合は、行スコープ候補から除外する。
2. 添字がなく、繰返しクラス祖先を持つ結合を候補とする。
3. 最も深い繰返しクラスを採用する。

このため、**Invoices_Received_Lines** は **dInvoiceLine** ごとに一行を出力します。一方、**Invoices_Received** は添字付きVAT内訳列を含む場合でも、請求書レベルの一行を維持します。

繰返し行スコープがない場合の変換

row_scope_group(...) が行スコープを返さない場合、**transform_rows(...)** は請求書レベルの出力行を作成します。

作業用変数：

- **output_rows:** list[dict[str, str]]
- **current_row:** dict[str, str] | None
- **repeat_counts:** dict[str, int]
- **key_binding:** SemanticBinding

スクリプトは構造化CSVの行を順に調べます。**row_has_bound_data(...)** は、対象出力に関係しない疎な行を除外します。**row_repeat_indices(...)** は、添字付きグループの出現番号を設定します。**merge_values(...)** が空でない値を現在の出力行へコピーします。

繰返し行スコープがある場合の変換

row_scope_group(...) がスコープを返した場合、**transform_repeated_group_rows(...)** は繰返し出現ごとに一つの出力行を生成します。

例：

```
scope_path=$.invoice.invoiceLine
scope_column=dInvoiceLine
```

作業用変数：

- **parent_context:** dict[str, str]：請求書レベルの値
- **current_group_row:** dict[str, str] | None：現在の繰返し出力行
- **current_scope_value:** str：現在の dXxx 出現値
- **parent_bindings:** list[SemanticBinding]
- **scoped_bindings:** list[SemanticBinding]

dlInvoice が設定され、選択した **scope_column** が空の行は **parent_context** を設定します。選択した **scope_column** が設定された行は、繰返し出力行を開始又は継続します。**scope_column** の値が変わると、現在の出力行を **output_rows** に追加し、**parent_context** を初期値として新しい行を開始します。

したがって、最初に値が入った繰返しディメンションから、最初の独立した出力レコードが始まります。親の値は **parent_context** からフラットな出力レコードへコピーします。入力構造化CSVの子行に親の値が重複していることは前提としません。この区別により、階層型入力の規則を保ちながら、自己完結したADS明細レコードを生成できます。

値の統合規則

merge_values(target_row, source_row, bindings, repeat_indices) だけが、出力行へ値を書き込みます。

規則：

1. **source_column** がない結合は処理しない。
2. **repeat_index** がある場合は、現在の出現番号が一致することを確認する。
3. **source_row[source_column]** を読み込む。
4. 入力が空の場合は **default_value** を使用する。
5. **target_row[target_column]** が空の場合だけ値を書き込む。

構造化CSVの行は疎で階層的であるため、この「最初の空でない値を採用する」規則が重要です。

セマンティック結合の出力及び統括処理

1. **row_has_bound_data(...)** が、選択した出力ビューに関係しない疎な行を除外する。
2. **row_scope_group(...)** が、請求書レベル変換又は添字のない最も深い繰返し出力スコープを選択する。
3. **row_repeat_indices(...)** と **repeated_group_present(...)** が、明示的な添字付きセマンティック結合で使用する0始まりの添字を管理する。
4. **transform_rows(...)** が、請求書レベルの統合又は **transform_repeated_group_rows(...)** を呼び分ける。
5. **write_target_file(...)** が、結合表の列順及び選択した区切り文字で出力する。
6. **output_format_options(...)** が、PSV又はCSVの既定値及びコマンドライン上書きを解決する。
7. **output_path(...)** と **binding_target_stem(...)** が、出力先及びファイル名を決定する。
8. **convert_file(...)** が一つの入力ファイルを変換し、**main(...)** が **input_files(...)** によりディレクトリ入力を展開する。